```
<!-- Q1. Create and manipulate DOM elements using getElementById(),
querySelector(), and querySelectorAll() -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>DOM Manipulation Lab</title>
    <style>
        body { font-family: sans-serif; padding: 20px; line-height: 1.6; }
        .highlight { background-color: yellow; padding: 5px; }
        .box { border: 2px solid black; margin-top: 10px; padding: 10px; width: fit-
content; }
    </style>
</head>
<body>

    <!-- Elements to be manipulated -->
    <h1 id="main-title">Original Title</h1>

    <p class="description">This is the first description paragraph.</p>
    <p class="description">This is the second description paragraph.</p>

    <ul>
        <li class="item">Item 1</li>
        <li class="item">Item 2</li>
        <li class="item">Item 3</li>
    </ul>

    <div id="container">
        <!-- New elements will be added here -->
    </div>

    <!-- JavaScript Code -->
    <script>
        // 1. getElementById()
        // Targets the unique ID 'main-title'
        const title = document.getElementById("main-title");
        title.textContent = "Title Updated via getElementById";
        title.style.color = "darkblue";

        // 2. querySelector()
        // Targets only the FIRST element with class 'description'
        const firstPara = document.querySelector(".description");
        firstPara.style.fontStyle = "italic";
        firstPara.innerHTML = "<strong>Updated via querySelector (First match
only)</strong>";
```

```
        // 3. querySelectorAll()
        // Targets ALL elements with class 'item'
        const allItems = document.querySelectorAll(".item");
        // Since it returns a NodeList, we use forEach to loop through them
        allItems.forEach((item, index) => {
            item.style.color = "green";
            item.textContent = "Updated Item " + (index + 1);
        });

        // 4. Creating and Appending a new element
        const container = document.getElementById("container");
        const newDiv = document.createElement("div");
        newDiv.className = "box";
        newDiv.textContent = "I was created using document.createElement()";

        container.appendChild(newDiv);

        console.log("DOM Manipulation complete!");
    </script>
</body>
</html>
```

```
<!-- Q2. Implement DOM traversal techniques and dynamically update content
using innerText, innerHTML, and textContent -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>DOM Traversal & Content Update</title>
    <style>
        .parent { border: 2px solid black; padding: 20px; background-color: #f0f0f0; }
        .child { margin: 10px; padding: 10px; border: 1px dashed red; background-
color: white; }
        .hidden { display: none; }
    </style>
</head>
<body>

    <div id="main-container" class="parent">
        Parent Division
        <div id="middle-child" class="child">
            Middle Child
            <span class="hidden"> (I am hidden text)</span>
        </div>
        <div class="child">Next Sibling Child</div>
```

```
    </div>

    <hr>
    <div id="output-area"></div>

    <script>
        // --- 1. DOM TRAVERSAL ---
        const middleChild = document.getElementById("middle-child");

        // Move UP: Get the parent
        const parent = middleChild.parentElement;
        parent.style.borderColor = "blue";

        // Move SIDEWAYS: Get the next sibling
        const nextChild = middleChild.nextElementSibling;
        nextChild.style.fontWeight = "bold";

        // Move DOWN: Get the parent's children
        const allChildren = parent.children; // Returns a collection
        console.log("Total children in parent:", allChildren.length);


        // --- 2. UPDATING CONTENT ---
        const output = document.getElementById("output-area");

        // A. innerHTML: Parses string as HTML (Renders tags)
        middleChild.innerHTML = "<h3>Updated with innerHTML</h3>";

        // B. innerText: Shows only visible text (ignores hidden spans)
        console.log("innerText view:", middleChild.innerText);

        // C. textContent: Shows all text (includes hidden spans)
        console.log("textContent view:", middleChild.textContent);

        // Demonstrating difference in the output area
        const demoDiv = document.createElement("div");

        // Example of innerHTML (renders the button)
        demoDiv.innerHTML = "<button>I am a rendered button</button>";
        output.appendChild(demoDiv);

        // Example of textContent (prints tags as raw text)
        const demoDiv2 = document.createElement("div");
        demoDiv2.textContent = "<button>I am just text, not a button</button>";
        output.appendChild(demoDiv2);

    </script>
</body>
```

```
</html>
```

```
<!-- Q3. Apply CSS styling dynamically using JavaScript and the style object  -->
 <!DOCTYPE html>
<html>
<head>
    <title>Dynamic Styling</title>
</head>
<body>

    <h1 id="text">Hello, Change My Style!</h1>

    <button onclick="changeStyle()">Apply Styles</button>
    <button onclick="resetStyle()">Reset</button>

    <script>
        const myText = document.getElementById("text");

        function changeStyle() {
            // Using the .style object
            myText.style.color = "white";
            myText.style.backgroundColor = "purple";
            myText.style.fontSize = "50px";
            myText.style.padding = "20px";
            myText.style.borderRadius = "15px";
            myText.style.textAlign = "center";
        }

        function resetStyle() {
            // Clearing styles
            myText.style.color = "";
            myText.style.backgroundColor = "";
            myText.style.fontSize = "32px";
            myText.style.padding = "0px";
        }
    </script>

</body>
</html>
```

```
<!-- Q4. Implement event handling for mouse and keyboard events using
addEventListener() and removeEventListener() -->
```

```html
<!DOCTYPE html>
<html>
<head>
    <title>Event Handling Lab</title>
    <style>
        #box {
            width: 200px;
            height: 200px;
            background-color: lightblue;
            display: flex;
            align-items: center;
            justify-content: center;
            border: 2px solid #333;
            margin-bottom: 10px;
        }
    </style>
</head>
<body>

    <h3>1. Mouse & Keyboard Events</h3>
    <div id="box">Hover or Click Me</div>
    <input type="text" id="keyInput" placeholder="Type something here...">

    <p id="message">Status: Waiting for action...</p>

    <hr>
    <h3>2. Add/Remove Listener</h3>
    <button id="stopBtn">Stop Box Interaction</button>

    <script>
        const box = document.getElementById("box");
        const keyInput = document.getElementById("keyInput");
        const message = document.getElementById("message");
        const stopBtn = document.getElementById("stopBtn");

        // --- NAMED FUNCTIONS (Required for removeEventListener) ---
        function handleMouseClick() {
            box.style.backgroundColor = "orange";
            message.textContent = "Status: Box Clicked!";
        }

        function handleMouseEnter() {
            box.style.backgroundColor = "lightgreen";
            message.textContent = "Status: Mouse Entered!";
        }

        function handleKeyDown(event) {
```

```
            message.textContent = "Status: You pressed the key: " + event.key;
        }

        // --- 1. ADDING EVENT LISTENERS ---
        // Mouse events
        box.addEventListener("click", handleMouseClick);
        box.addEventListener("mouseenter", handleMouseEnter);

        // Keyboard event (attached to the input field)
        keyInput.addEventListener("keydown", handleKeyDown);

        // --- 2. REMOVING EVENT LISTENERS ---
        stopBtn.addEventListener("click", () => {
            // This removes the click and hover functionality from the box
            box.removeEventListener("click", handleMouseClick);
            box.removeEventListener("mouseenter", handleMouseEnter);

            box.style.backgroundColor = "gray";
            box.textContent = "Disabled";
            message.textContent = "Status: Event Listeners Removed!";
        });
    </script>

</body>
</html>
```

```
<!-- Q5.  Demonstrate event propagation (bubbling and capturing) and prevent
default form submission behavior -->
<!DOCTYPE html>
<html>
<head>
    <title>Event Propagation Lab</title>
    <style>
        div { padding: 30px; border: 2px solid black; cursor: pointer; }
        #grandparent { background-color: lightcoral; }
        #parent { background-color: lightblue; }
        #child { background-color: lightgreen; padding: 15px; border: 1px solid black;
}
        .form-section { margin-top: 30px; padding: 10px; border-top: 2px dashed gray;
}
    </style>
</head>
<body>

    <h3>1. Event Bubbling & Capturing</h3>
```

```
<p>Check the Browser Console (F12) to see the order of events!</p>

<div id="grandparent">
    Grandparent
    <div id="parent">
        Parent
        <div id="child">Child (Click Me)</div>
    </div>
</div>

<div class="form-section">
    <h3>2. Prevent Default Behavior</h3>
    <form id="myForm">
        <input type="text" placeholder="Type something...">
        <button type="submit">Submit Form</button>
    </form>
    <p id="form-msg"></p>
</div>

<script>
    const gp = document.getElementById("grandparent");
    const p = document.getElementById("parent");
    const c = document.getElementById("child");

    // --- BUBBLING (Default: False) ---
    // Order: Child -> Parent -> Grandparent
    c.addEventListener("click", (e) => {
        console.log("Child Clicked (Bubbling)");
        // e.stopPropagation(); // Uncomment this to stop the event here
    }, false);

    p.addEventListener("click", () => {
        console.log("Parent Clicked (Bubbling)");
    }, false);

    gp.addEventListener("click", () => {
        console.log("Grandparent Clicked (Bubbling)");
    }, false);


    // --- CAPTURING (Set third parameter to True) ---
    // Order: Grandparent -> Parent -> Child
    /*
    gp.addEventListener("click", () => {
        console.log("Grandparent Clicked (Capturing)");
    }, true);
    */
```

```
        // --- PREVENT DEFAULT ---
        const myForm = document.getElementById("myForm");
        const formMsg = document.getElementById("form-msg");

        myForm.addEventListener("submit", (e) => {
            e.preventDefault(); // STOPS the page from refreshing
            formMsg.textContent = "Form submitted! But page did NOT refresh thanks to
preventDefault().";
            formMsg.style.color = "blue";
        });
    </script>
</body>
</html>
```

```
<!-- Q6. Compare synchronous and asynchronous execution using setTimeout()
and setInterval() -->
<!DOCTYPE html>
<html>
<head>
    <title>Sync vs Async Lab</title>
    <style>
        body { font-family: sans-serif; line-height: 1.6; }
        .box { border: 1px solid #ccc; padding: 10px; margin: 10px 0; background:
#f9f9f9; }
        #counter-display { font-size: 24px; color: blue; font-weight: bold; }
    </style>
</head>
<body>

    <h2>Synchronous vs Asynchronous Execution</h2>
    <button onclick="runComparison()">Run Sync vs Async Demo</button>
    <div class="box" id="log-output">Check the Console (F12) to see execution
order.</div>

    <hr>

    <h2>setInterval Demo (Dynamic Counter)</h2>
    <div class="box">
        <p>Interval Count: <span id="counter-display">0</span></p>
        <button onclick="startCounter()">Start Counter</button>
        <button onclick="stopCounter()">Stop Counter</button>
    </div>

    <script>
        // --- 1. Synchronous vs Asynchronous ---
```

```
        function runComparison() {
            console.log("--- Start of Comparison ---");

            // Synchronous Task
            console.log("1. I am Synchronous (Step 1)");

            // Asynchronous Task (setTimeout)
            setTimeout(() => {
                console.log("2. I am Asynchronous (Step 2 - Delayed by 2s)");
            }, 2000);

            // Synchronous Task
            console.log("3. I am Synchronous (Step 3)");

            alert("Check console: Step 3 appeared before Step 2 because Step 2 is
Asynchronous!");
        }

        // --- 2. setInterval Demo ---
        let timerId;
        let count = 0;

        function startCounter() {
            // Check if timer is already running to avoid duplicates
            if (timerId) return;

            timerId = setInterval(() => {
                count++;
                document.getElementById("counter-display").textContent = count;
                console.log("Interval running... " + count);
            }, 1000); // Runs every 1 second
        }

        function stopCounter() {
            clearInterval(timerId); // Stops the interval
            timerId = null;
            console.log("Interval Stopped.");
        }
    </script>
</body>
</html>
```

```
<!-- Q7. Implement callback functions and analyze callback hell with nested
asynchronous calls -->

<!DOCTYPE html>
<html>
<head>
    <title>Callback Hell Lab</title>
</head>
<body>

    <h2>Callback & Callback Hell Demo</h2>
    <button onclick="startProcess()">Start Pizza Order (Callback Hell)</button>
    <p id="status">Check console for process details...</p>

    <script>
        // Simulating a Pizza Ordering System

        function orderPizza(callback) {
            setTimeout(() => {
                console.log("1. Pizza Ordered");
                callback();
            }, 1000);
        }

        function preparePizza(callback) {
            setTimeout(() => {
                console.log("2. Pizza Prepared");
                callback();
            }, 1000);
        }

        function deliverPizza(callback) {
            setTimeout(() => {
                console.log("3. Pizza Delivered");
                callback();
            }, 1000);
        }

        // --- CALLBACK HELL (The Pyramid of Doom) ---
        function startProcess() {
            document.getElementById("status").textContent = "Process started... check
console.";

            // Nested callbacks start here
            orderPizza(() => {
                preparePizza(() => {
                    deliverPizza(() => {
                        console.log("All steps complete! Enjoy your pizza.");
```

```
                        document.getElementById("status").textContent = "Pizza
Delivered!";
                    });
                });
            });
        }
    </script>
</body>
</html>
```

```
<!-- Q8. Develop asynchronous workflows using Promises with then(), catch(),
and finally() -->

<!DOCTYPE html>
<html>
<head>
    <title>Promises Lab</title>
</head>
<body>

    <h2>JavaScript Promise Workflow</h2>
    <button onclick="checkOrder()">Check Order Status</button>

    <p id="msg">Click the button to start...</p>
    <p id="status" style="font-weight: bold;"></p>

    <script>
        function checkOrder() {
            const msg = document.getElementById("msg");
            const status = document.getElementById("status");

            msg.textContent = "Checking with server... please wait.";
            status.textContent = "";

            // 1. Creating a Promise
            const myPromise = new Promise((resolve, reject) => {
                let isStockAvailable = Math.random() > 0.5; // Randomly true or false

                setTimeout(() => {
                    if (isStockAvailable) {
                        resolve("Success: Item is in stock!");
                    } else {
                        reject("Error: Item is out of stock!");
                    }
                }, 2000);
```

```
            });

            // 2. Handling the Promise
            myPromise
                .then((result) => {
                    // Runs if resolve() is called
                    status.style.color = "green";
                    status.textContent = result;
                })
                .catch((error) => {
                    // Runs if reject() is called
                    status.style.color = "red";
                    status.textContent = error;
                })
                .finally(() => {
                    // Runs no matter what (resolved or rejected)
                    msg.textContent = "Process Finished.";
                    console.log("Operation completed.");
                });
        }
    </script>
</body>
</html>
```

```
<!-- Q9. Fetch and display data from a public API using async/await and Fetch
API -->
<!DOCTYPE html>
<html>
<head>
    <title>Fetch API with Async/Await</title>
    <style>
        body { font-family: Arial, sans-serif; padding: 20px; }
        .user-card {
            border: 1px solid #ddd;
            padding: 10px;
            margin: 10px 0;
            border-radius: 5px;
            background-color: #f9f9f9;
        }
        #loading { color: blue; font-weight: bold; }
    </style>
</head>
<body>

    <h2>User List from Public API</h2>
```

```
    <button onclick="getUsers()">Fetch Users</button>

    <div id="loading" style="display: none;">Loading data...</div>
    <div id="user-container"></div>

    <script>
        // async keyword allows the use of 'await' inside the function
        async function getUsers() {
            const container = document.getElementById("user-container");
            const loader = document.getElementById("loading");

            // Show loader and clear old data
            loader.style.display = "block";
            container.innerHTML = "";

            try {
                // await pauses the function until the fetch Promise resolves
                const response = await
fetch("https://jsonplaceholder.typicode.com/users");

                // Check if the response is okay (status 200)
                if (!response.ok) {
                    throw new Error("Network response was not ok");
                }

                // Convert response to JSON
                const users = await response.json();

                // Display data
                users.forEach(user => {
                    const div = document.createElement("div");
                    div.className = "user-card";
                    div.innerHTML = `
                        <strong>Name:</strong> ${user.name} <br>
                        <strong>Email:</strong> ${user.email} <br>
                        <strong>City:</strong> ${user.address.city}
                    `;
                    container.appendChild(div);
                });

            } catch (error) {
                console.error("Error fetching data:", error);
                container.innerHTML = "<p style='color:red;'>Failed to load
data.</p>";
            } finally {
                // Hide loader regardless of success or failure
                loader.style.display = "none";
            }
```

```
        }
    </script>

</body>
</html>
```

```
<!-- Q10. Store and retrieve application data using Local Storage and Session
Storage -->
<!DOCTYPE html>
<html>
<head>
    <title>Storage Lab</title>
    <style>
        body { font-family: sans-serif; padding: 20px; }
        .box { border: 1px solid #ccc; padding: 15px; margin-bottom: 20px; border-
radius: 8px; }
        input { padding: 8px; margin: 5px; }
        button { padding: 8px 12px; cursor: pointer; }
    </style>
</head>
<body>

    <h2>Web Storage API Demo</h2>

    <!-- Local Storage Section -->
    <div class="box">
        <h3>1. Local Storage (Persistent)</h3>
        <p>Data stays even if you close the browser.</p>
        <input type="text" id="localInput" placeholder="Enter name...">
        <button onclick="saveLocal()">Save to Local</button>
        <button onclick="loadLocal()">Retrieve</button>
        <p id="localDisplay"></p>
    </div>

    <!-- Session Storage Section -->
    <div class="box">
        <h3>2. Session Storage (Temporary)</h3>
        <p>Data clears when you close the tab.</p>
        <input type="text" id="sessionInput" placeholder="Enter session info...">
        <button onclick="saveSession()">Save to Session</button>
        <button onclick="loadSession()">Retrieve</button>
        <p id="sessionDisplay"></p>
    </div>

    <button onclick="clearAll()">Clear All Storage</button>
```

```
    <script>
        // --- LOCAL STORAGE ---
        function saveLocal() {
            const val = document.getElementById("localInput").value;
            localStorage.setItem("username", val);
            alert("Saved to Local Storage!");
        }

        function loadLocal() {
            const data = localStorage.getItem("username");
            document.getElementById("localDisplay").textContent = "Stored Name: " +
(data || "No data found");
        }

        // --- SESSION STORAGE ---
        function saveSession() {
            const val = document.getElementById("sessionInput").value;
            sessionStorage.setItem("sessionID", val);
            alert("Saved to Session Storage!");
        }

        function loadSession() {
            const data = sessionStorage.getItem("sessionID");
            document.getElementById("sessionDisplay").textContent = "Stored Session: "
+ (data || "No data found");
        }

        // --- CLEARING DATA ---
        function clearAll() {
            localStorage.clear();
            sessionStorage.clear();
            alert("All storage cleared!");
            location.reload(); // Refresh page to see changes
        }
    </script>
</body>
</html>
```

```
// Q11. Create a React application using Vite and implement JSX expressions
// 1. Create the project
// npm create vite@latest my-react-app -- --template react

// 2. Go into the folder
// cd my-react-app
```

```jsx
// 3. Install dependencies
// npm install

// 4. Start the development server
// npm run dev

import React from 'react';

function App() {
  // 1. Defining variables
  const name = "Rahul";
  const course = "Web Development";
  const year = 2027;
  const isLoggedIn = true;

  // 2. Defining a function to be used in JSX
  const getGreeting = (user) => {
    return `Welcome back, ${user}!`;
  };

  return (
    <div style={{ padding: '20px', fontFamily: 'Arial' }}>
      <h1>React with Vite</h1>

      {/* Using curly braces {} to inject JS expressions */}
      <p><strong>Student Name:</strong> {name}</p>
      <p><strong>Course:</strong> {course}</p>

      {/* Arithmetic Expression */}
      <p><strong>Next Year will be:</strong> {year + 1}</p>

      {/* Function Call Expression */}
      <h3>{getGreeting(name)}</h3>

      {/* Ternary Operator (Conditional Expression) */}
      <div>
        {isLoggedIn ? (
          <p style={{ color: 'green' }}>Status: You are logged in.</p>
        ) : (
          <p style={{ color: 'red' }}>Status: Please log in.</p>
        )}
      </div>

      {/* Using JS logic for dynamic attributes */}
      <button disabled={!isLoggedIn}>Access Dashboard</button>
    </div>
  );
```

```
}

export default App;
```

```
// Q12. Develop functional components and pass data using props

import StudentCard from "../StudentCard";
function App() {
  return (
    <div>
      <h1>College Directory</h1>

      {/* Passing data using props */}
      <StudentCard
        name="Amit Sharma"
        roll={101}
        course="Computer Science"
      />

      <StudentCard
        name="Sneha Kapoor"
        roll={102}
        course="Information Technology"
      />
    </div>
  );
}

export default App;

// StudentCard.jsx

// We use 'destructuring' { name, roll, course } to get values directly
function StudentCard({ name, roll, course }) {
  return (
    <div style={{ border: '1px solid black', padding: '10px', margin: '10px',
borderRadius: '8px' }}>
      <h2>Student Details</h2>
      <p><strong>Name:</strong> {name}</p>
      <p><strong>Roll No:</strong> {roll}</p>
      <p><strong>Course:</strong> {course}</p>
    </div>
  );
}
```

```
export default StudentCard;
```

```
// Q13. Manage component state and conditional rendering in React
import { useState } from "react";

function App() {
  // 1. Create a state variable (isVisible) with initial value 'false'
  const [isVisible, setIsVisible] = useState(false);

  return (
    <div style={{ padding: "20px", textAlign: "center" }}>
      <h1>React Easy State Demo</h1>

      {/* 2. Conditional Rendering: Only show the <h2> if isVisible is TRUE */}
      {isVisible && <h2 style={{ color: "blue" }}>Hello! I am now visible.</h2>}

      {/* 3. Button to toggle the state */}
      <button onClick={() => setIsVisible(!isVisible)}>
        {/* Ternary logic to change button text based on state */}
        {isVisible ? "Hide Message" : "Show Message"}
      </button>
    </div>
  );
}

export default App;
```

```
// Q14. Implement component lifecycle behavior using React hooks or lifecycle methods

import { useState, useEffect } from "react";

// This is a separate component to demonstrate "Unmounting"
function TimerComponent() {
  useEffect(() => {
    // 1. MOUNTING: Runs once when the component appears
    console.log("Timer Component: Mounted (Created)");

    // 3. UNMOUNTING: The 'cleanup' function runs when the component disappears
    return () => {
      console.log("Timer Component: Unmounted (Destroyed)");
    };
  }, []);

  return <h2 style={{ color: "red" }}>The Timer is running...</h2>;
```

```
}

function App() {
  const [count, setCount] = useState(0);
  const [showTimer, setShowTimer] = useState(true);

  // 2. UPDATING: Runs every time the 'count' state changes
  useEffect(() => {
    if (count > 0) {
      console.log("App Component: Updated! New count is " + count);
    }
  }, [count]); // Dependency array: only runs when 'count' changes

  return (
    <div style={{ padding: "20px", textAlign: "center" }}>
      <h1>React Lifecycle Demo</h1>
      <p>Check the **Console (F12)** to see the lifecycle logs!</p>

      {/* Button to trigger Update */}
      <button onClick={() => setCount(count + 1)}>
        Update Count: {count}
      </button>

      <hr />

      {/* Button to trigger Mount/Unmount */}
      <button onClick={() => setShowTimer(!showTimer)}>
        {showTimer ? "Remove Timer Component" : "Add Timer Component"}
      </button>

      {showTimer && <TimerComponent />}
    </div>
  );
}

export default App;
```

```
// Q15. Apply different styling techniques in React including CSS Modules and Tailwind
CSS

import React from 'react';
// 1. Importing CSS Module
import styles from './MyStyle.module.css';

function App() {
```

```
  // 2. Defining Inline Styles (Object)
  const inlineBox = {
    backgroundColor: 'lightblue',
    padding: '20px',
    borderRadius: '10px',
    border: '2px solid blue',
    marginBottom: '10px'
  };

  return (
    <div style={{ padding: '20px' }}>
      <h1>React Styling Techniques</h1>

      {/* Technique 1: Inline Styling */}
      <div style={inlineBox}>
        <strong>1. Inline Style:</strong> Styled using a JS Object.
      </div>

      {/* Technique 2: CSS Modules */}
      <div className={styles.moduleBox}>
        <strong>2. CSS Module:</strong> Styled using an imported local class.
      </div>

      {/* Technique 3: Tailwind CSS (Utility Classes) */}
      {/* Note: Tailwind must be installed/configured in your project to work */}
      <div className="bg-purple-500 text-white p-5 rounded-lg border-2 border-purple-
800">
        <strong>3. Tailwind CSS:</strong> Styled using utility classes like 'p-5' and
'bg-purple-500'.
      </div>

    </div>
  );
}

export default App;
// MyStyle.module.css
/* This class is scoped only to the component that imports it */

// .moduleBox {
//     background-color: lightgreen;
//     padding: 20px;
//     border-radius: 10px;
//     border: 2px solid green;
//     margin-bottom: 10px;
// }
```

```
// npm install -D tailwindcss postcss autoprefixer
// npx tailwindcss init -p

// Then add the paths to all of your template files in the tailwind.config.js file:
// module.exports = {
//   content: [
//     "./index.html",
//     "./src/**/*.{js,ts,jsx,tsx}",
//   ],
//   theme: {
//     extend: {},
//   },
//   plugins: [],
// }
// in css file add the following lines
// @tailwind base;
// @tailwind components;
// @tailwind utilities;
```

```
// Q16. Implement useState and useEffect hooks to manage state and side effects

import { useState, useEffect } from "react";

function App() {
  // 1. Manage State (using useState)
  const [count, setCount] = useState(0);

  // 2. Manage Side Effects (using useEffect)
  useEffect(() => {
    // This side effect updates the Browser Tab Title
    document.title = `Count: ${count}`;

    console.log("Effect triggered: The count is now " + count);

  }, [count]); // Dependency Array: Run this effect only when 'count' changes

  return (
    <div style={{ padding: "20px", textAlign: "center" }}>
      <h1>Hooks Demo</h1>

      <div style={{ fontSize: "24px", margin: "20px" }}>
        Current Count: <strong>{count}</strong>
      </div>
```

```
      <button
        onClick={() => setCount(count + 1)}
        style={{ padding: "10px 20px", fontSize: "16px", cursor: "pointer" }}
      >
        Increment Count
      </button>

      <p>Check the Browser Tab Title and Console (F12)!</p>
    </div>
  );
}

export default App;
```

```
// 17. Develop controlled forms with validation and submission handling in React

import { useState } from "react";

function App() {
  // 1. State for form inputs
  const [username, setUsername] = useState("");
  const [email, setEmail] = useState("");

  // 2. State for validation errors
  const [error, setError] = useState("");

  // 3. Handle Form Submission
  const handleSubmit = (e) => {
    e.preventDefault(); // Prevents page reload

    // Basic Validation Logic
    if (username.length < 3) {
      setError("Username must be at least 3 characters long.");
      return;
    }
    if (!email.includes("@")) {
      setError("Please enter a valid email address.");
      return;
    }

    // If validation passes
    setError(""); // Clear errors
    alert(`Form Submitted!\nName: ${username}\nEmail: ${email}`);

    // Optional: Reset form
```

```
    setUsername("");
    setEmail("");
  };

  return (
    <div style={{ padding: "20px", maxWidth: "400px" }}>
      <h2>Controlled Form</h2>

      <form onSubmit={handleSubmit}>
        {/* Username Input */}
        <div style={{ marginBottom: "10px" }}>
          <label>Username: </label> <br />
          <input
            type="text"
            value={username}
            onChange={(e) => setUsername(e.target.value)}
          />
        </div>

        {/* Email Input */}
        <div style={{ marginBottom: "10px" }}>
          <label>Email: </label> <br />
          <input
            type="email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
          />
        </div>

        {/* Display Validation Error */}
        {error && <p style={{ color: "red", fontSize: "14px" }}>{error}</p>}

        <button type="submit">Register</button>
      </form>
    </div>
  );
}

export default App;
```

```
// 18. Configure client-side routing using React Router with dynamic route parameters

// npm install react-router-dom

import { BrowserRouter, Routes, Route, Link, useParams } from "react-router-dom";
```

```jsx
// 1. HOME COMPONENT
function Home() {
  return (
    <div>
      <h2>Welcome to the Home Page</h2>
      <p>Select a user to view their profile:</p>
      <ul>
        <li><Link to="/user/101">View User 101</Link></li>
        <li><Link to="/user/202">View User 202</Link></li>
        <li><Link to="/user/Alpha">View User Alpha</Link></li>
      </ul>
    </div>
  );
}

// 2. DYNAMIC USER DETAIL COMPONENT
function UserProfile() {
  // useParams hook captures the dynamic ":id" from the URL
  const { id } = useParams();

  return (
    <div style={{ padding: "10px", border: "1px solid blue" }}>
      <h3>User Profile Page</h3>
      <p>Current User ID is: <strong style={{ color: "red" }}>{id}</strong></p>
      <Link to="/">Back to Home</Link>
    </div>
  );
}

// 3. MAIN APP (Router Configuration)
function App() {
  return (
    <BrowserRouter>
      <nav style={{ padding: "10px", backgroundColor: "#eee" }}>
        <Link to="/" style={{ marginRight: "10px" }}>Home</Link>
      </nav>

      <div style={{ padding: "20px" }}>
        <Routes>
          <Route path="/" element={<Home />} />

          {/* ":id" is a dynamic parameter. It can be anything. */}
          <Route path="/user/:id" element={<UserProfile />} />
        </Routes>
      </div>
    </BrowserRouter>
  );
```

```
}

export default App;
```

```
/ 19.Implement global state management using Context API and useContext hook

import { createContext, useState, useContext } from "react";

// 1. CREATE the Context (Like a global storage box)
const UserContext = createContext();

function App() {
  const [user, setUser] = useState("Rahul Kumar");

  return (
    // 2. PROVIDE the Context to all children
    <UserContext.Provider value={user}>
      <div style={{ padding: "20px", border: "2px solid black" }}>
        <h1>Parent Component (App)</h1>
        <p>User in App state: {user}</p>
        <hr />
        <MiddleComponent />
      </div>
    </UserContext.Provider>
  );
}

// A middle component that DOES NOT receive props
function MiddleComponent() {
  return (
    <div style={{ padding: "10px", backgroundColor: "#eee" }}>
      <h3>Middle Component</h3>
      <p>I am just a bridge. I don't use the user data.</p>
      <DeepChild />
    </div>
  );
}

// 3. CONSUME the Context in a deep child
function DeepChild() {
  // Use the useContext hook to reach up and grab the 'user' value
  const loggedInUser = useContext(UserContext);

  return (
```

```
    <div style={{ padding: "10px", backgroundColor: "lightyellow", border: "1px solid
gold" }}>
      <h4>Deep Child Component</h4>
      <p>Data received from Context: <strong>{loggedInUser}</strong></p>
    </div>
  );
}

export default App;
```

```
// Q20. Integrate REST APIs using Fetch or Axios with loading and error handling
import { useState, useEffect } from "react";

function App() {
  const [users, setUsers] = useState([]);    // To store API data
  const [loading, setLoading] = useState(true); // To track loading status
  const [error, setError] = useState(null);   // To store error messages

  useEffect(() => {
    // Define the async function
    const fetchData = async () => {
      try {
        setLoading(true); // Start loading

        const response = await fetch("https://jsonplaceholder.typicode.com/users");

        // If API returns an error (like 404), manually throw an error
        if (!response.ok) {
          throw new Error("Failed to fetch data from server.");
        }

        const data = await response.json();
        setUsers(data); // Save data to state
      } catch (err) {
        setError(err.message); // Save error message to state
      } finally {
        setLoading(false); // Stop loading regardless of success or failure
      }
    };

    fetchData();
  }, []); // Run only once on mount

  // 1. CONDITIONAL RENDERING FOR LOADING
```

```
  if (loading) return <h2 style={{ color: "blue" }}>Loading users... Please
wait.</h2>;

  // 2. CONDITIONAL RENDERING FOR ERROR
  if (error) return <h2 style={{ color: "red" }}>Error: {error}</h2>;

  // 3. RENDER DATA IF SUCCESSFUL
  return (
    <div style={{ padding: "20px" }}>
      <h1>User Directory (API Integration)</h1>
      <ul>
        {users.map((user) => (
          <li key={user.id} style={{ margin: "10px 0" }}>
            <strong>{user.name}</strong> - {user.email}
          </li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

```
// 21. Implement centralized state management using Redux Toolkit in a React
application

// npm install @reduxjs/toolkit react-redux

import React from "react";
import { configureStore, createSlice } from "@reduxjs/toolkit";
import { Provider, useSelector, useDispatch } from "react-redux";

// --- 1. CREATE A SLICE (Logic for a specific feature) ---
const counterSlice = createSlice({
  name: "counter",
  initialState: { value: 0 },
  reducers: {
    increment: (state) => { state.value += 1 },
    decrement: (state) => { state.value -= 1 },
  },
});

// Export the actions created by the slice
const { increment, decrement } = counterSlice.actions;
```

```jsx
// --- 2. CONFIGURE THE STORE (The Global Brain) ---
const store = configureStore({
  reducer: {
    counter: counterSlice.reducer,
  },
});

// --- 3. THE COMPONENT ---
function Counter() {
  // useSelector gets data from the store
  const count = useSelector((state) => state.counter.value);
  // useDispatch sends actions to the store
  const dispatch = useDispatch();

  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h2>Redux Toolkit Counter</h2>
      <h1>{count}</h1>
      <button onClick={() => dispatch(increment())}>Increment</button>
      <button onClick={() => dispatch(decrement())} style={{ marginLeft: "10px" }}>
        Decrement
      </button>
    </div>
  );
}

// --- 4. THE MAIN APP (Providing the Store) ---
function App() {
  return (
    // The Provider makes the store available to all components
    <Provider store={store}>
      <Counter />
    </Provider>
  );
}

export default App;
```

```jsx
// 22. Optimize application performance using lazy loading and code splitting in React

import React, { useState, lazy, Suspense } from "react";

// 1. LAZY LOADING: Import the component using React.lazy
// This tells React: "Don't download this file until we actually need it."
const HeavyComponent = lazy(() => import("../HeavyComponent"));

function App() {
  const [show, setShow] = useState(false);

  return (
    <div style={{ padding: "20px", textAlign: "center" }}>
      <h1>Performance Optimization</h1>
      <p>Initial bundle is small because HeavyComponent isn't included yet.</p>

      <button onClick={() => setShow(true)}>
        Load Heavy Component
      </button>

      {show && (
        // 2. SUSPENSE: This is required when using lazy loading.
        // The 'fallback' is what the user sees while the component is downloading.
        <Suspense fallback={<h3 style={{ color: "orange" }}>Loading Chunks...</h3>}>
          <HeavyComponent />
        </Suspense>
      )}
    </div>
  );
}

export default App;
// HeavyComponent.jsx
export default function HeavyComponent() {
  return (
    <div style={{ padding: "20px", backgroundColor: "#f0f0f0", marginTop: "20px" }}>
      <h2>I am the Heavy Component!</h2>
      <p>I was loaded only after you clicked the button.</p>
    </div>
  );
}
```

```jsx
// 23. Mini capstone with CRUD

import React, { useState, useEffect } from "react";

function App() {
  const [products, setProducts] = useState([]); // READ from API
  const [cart, setCart] = useState([]);         // CRUD on Cart
  const [loading, setLoading] = useState(true);

  // 1. READ: Fetching Fake Products from API
  useEffect(() => {
    fetch("https://fakestoreapi.com/products")
      .then((res) => res.json())
      .then((data) => {
        setProducts(data);
        setLoading(false);
      });
  }, []);

  // 2. CREATE: Add Item to Cart
  const addToCart = (product) => {
    const existing = cart.find((item) => item.id === product.id);
    if (existing) {
      // 3. UPDATE: If item exists, increase quantity
      setCart(cart.map((item) =>
        item.id === product.id ? { ...item, qty: item.qty + 1 } : item
      ));
    } else {
      setCart([...cart, { ...product, qty: 1 }]);
    }
  };

  // 4. DELETE: Remove Item from Cart
  const removeFromCart = (id) => {
    setCart(cart.filter((item) => item.id !== id));
  };

  if (loading) return <div className="text-center mt-20 text-2xl">Loading Amazon
Store...</div>;

  return (
    <div className="bg-gray-100 min-h-screen">
      {/* RESPONSIVE NAVBAR */}
      <nav className="bg-slate-900 text-white p-4 sticky top-0 z-50 flex justify-
between items-center px-4 md:px-10">
        <h1 className="text-2xl font-bold text-orange-400">amazon.in</h1>
        <div className="relative">
          <span className="font-bold">Cart 🛒</span>
```

```
          <span className="absolute -top-2 -right-3 bg-orange-500 text-xs rounded-full
px-1.5">{cart.length}</span>
        </div>
      </nav>

      <div className="max-w-7xl mx-auto p-4 flex flex-col md:flex-row gap-6">

        {/* PRODUCT GRID (READ) */}
        <div className="md:w-3/4 grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-
4">
          {products.map((product) => (
            <div key={product.id} className="bg-white p-4 rounded shadow flex flex-col
justify-between">
              <img src={product.image} alt={product.title} className="h-40 object-
contain mb-4" />
              <h2 className="font-bold text-sm line-clamp-2">{product.title}</h2>
              <p className="text-orange-700 font-bold text-xl my-
2">${product.price}</p>
              <button
                onClick={() => addToCart(product)}
                className="w-full bg-yellow-400 hover:bg-yellow-500 py-2 rounded-md
font-medium transition">
                Add to Cart
              </button>
            </div>
          ))}
        </div>

        {/* SIDEBAR CART (CRUD Display) */}
        <div className="md:w-1/4 bg-white p-4 rounded shadow h-fit sticky top-20">
          <h2 className="text-xl font-bold border-b pb-2 mb-4">Your Shopping
Basket</h2>
          {cart.length === 0 ? <p>Your cart is empty.</p> : (
            cart.map(item => (
              <div key={item.id} className="flex justify-between items-center mb-4
border-b pb-2">
                <div className="text-xs">
                  <p className="font-bold line-clamp-1">{item.title}</p>
                  <p>Qty: {item.qty} | ${item.price}</p>
                </div>
                <button
                  onClick={() => removeFromCart(item.id)}
                  className="text-red-500 font-bold text-xs">Remove</button>
              </div>
            ))
          )}
          <div className="mt-4 font-bold text-lg">
```

```
            Total: ${cart.reduce((acc, curr) => acc + curr.price * curr.qty,
0).toFixed(2)}
          </div>
        </div>

      </div>
    </div>
  );
}

export default App;
```